Réplication et partitionnement MongoDB

Compte rendu du projet M201 | Comprendre les bases de données NoSQL

Groupe : Ali Bouyakhsass · Fatima Zahra Bayouni · Khalid El Hamraoui | Formateur : Abderrahman SALHI | Date : [16/10/2026]

1. Introduction

Une plateforme de commerce en ligne connaît une croissance rapide. Sa base de données MongoDB tourne aujourd'hui sur un serveur unique : si celui-ci tombe, le site devient inaccessible et des commandes sont perdues. Ce serveur est donc un point de défaillance unique, et il ne pourra pas absorber indéfiniment la croissance du volume de données.

MongoDB propose deux mécanismes complémentaires pour répondre à ces deux problèmes. La **réplication** maintient plusieurs copies des mêmes données sur plusieurs serveurs (un jeu de réplicas) : elle apporte la haute disponibilité. Le **partitionnement** (sharding) répartit les données d'une collection sur plusieurs serveurs : il apporte le passage à l'échelle.

Nous avons mis en œuvre la réplication avec un jeu de réplicas de trois instances lancées avec Docker Compose, puis nous l'avons mis à l'épreuve en arrêtant le primaire. Le partitionnement est traité à l'écrit (section 5). Nous terminons par une recommandation pour la plateforme (section 7).

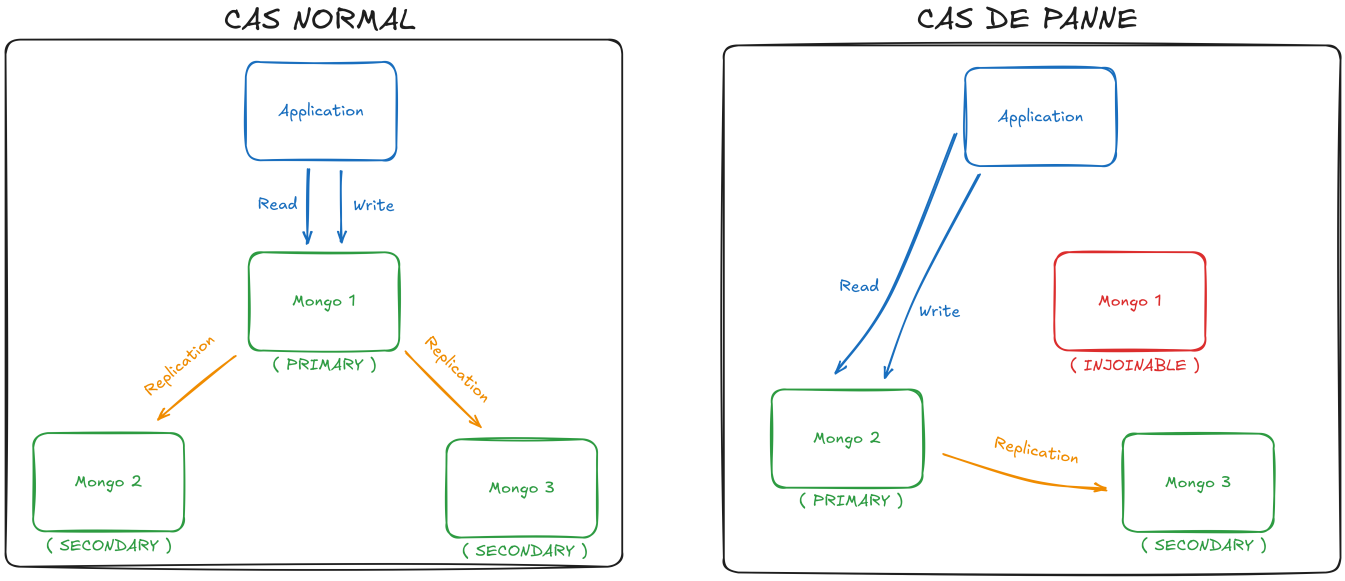
2. Architecture mise en œuvre

Nous avons utilisé Docker Compose pour lancer trois conteneurs basés sur l'image officielle `mongo:7`, sur un même réseau Docker (`mongo-net`). Chaque instance est démarrée avec l'option `--replSet rs0` : elle sait qu'elle fera partie du jeu de réplicas nommé `rs0`, mais ce n'est qu'à l'étape de configuration (`rs.initiate`) que les trois instances sont réellement reliées. Chaque instance dispose de son propre volume pour conserver ses données.

Instance	Port exposé sur l'hôte	Rôle au départ	Rôle après la panne (essai n°1)	Rôle après redémarrage
mongo1	27017	PRIMARY	arrêtée (`docker kill`)	SECONDARY

Instance	Port exposé sur l'hôte	Rôle au départ	Rôle après la panne (essai n°1)	Rôle après redémarrage
mongo2	27018	SECONDARY	PRIMARY (élue)	PRIMARY
mongo3	27019	SECONDARY	SECONDARY	SECONDARY

> **SCHÉMA DE L'ARCHITECTURE**



Fichier `docker-compose.yml` utilisé (extrait, les trois services sont identiques à leur nom et à leur port près) :

```
mongo1:
  image: mongo:7
  container_name: mongo1
  command: ["mongod", "--replSet", "rs0", "--bind_ip_all"]
  ports: ["27017:27017"]
  volumes: ["mongo1-data:/data/db"]
  networks: [mongo-net]
```

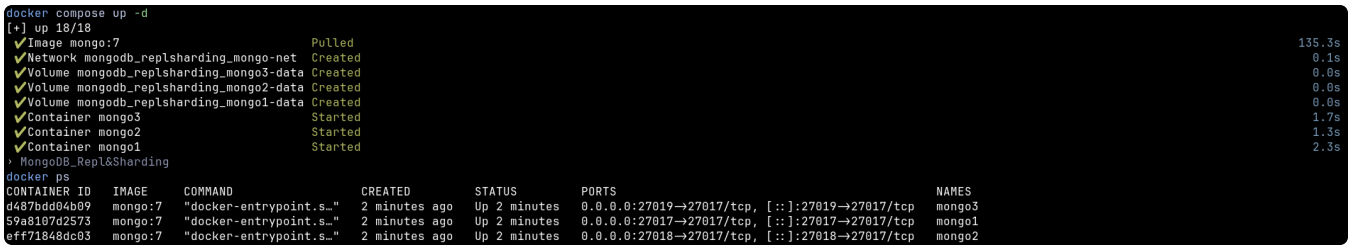
3. Déroulement de la simulation

Toutes les commandes ont été exécutées dans un terminal, via `docker exec`, qui ouvre un shell `mongosh` à l'intérieur du conteneur choisi.

3.1 Lancement des trois instances

Commande :

```
docker compose up -d
docker ps
```



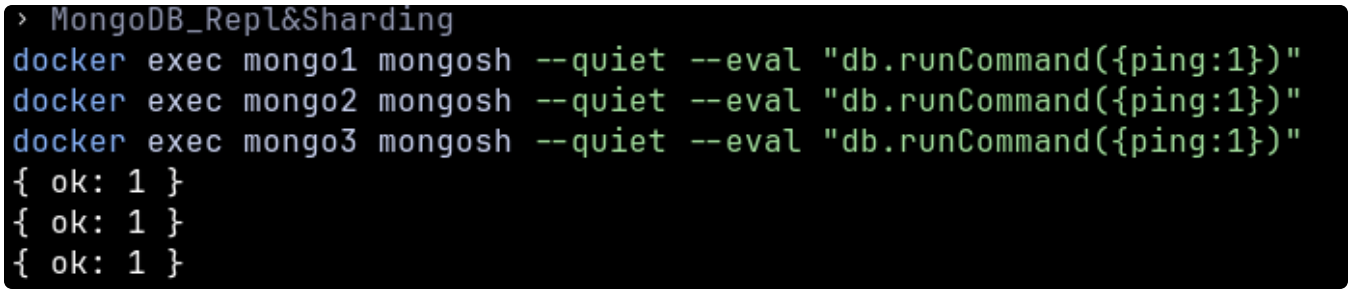
Capture C1: `docker compose up -d` puis `docker ps` : les trois conteneurs mongo1, mongo2 et mongo3 sont à l'état « Up ».

Explication : `docker compose up -d` lit le fichier `docker-compose.yml`, télécharge l'image si nécessaire et démarre les trois conteneurs en arrière-plan. `docker ps` confirme que les trois processus `mongod` tournent. À ce stade, ce sont encore trois serveurs indépendants, qui ne se connaissent pas.

3.2 Vérification que chaque instance répond

Commande :

```
docker exec mongo1 mongosh --quiet --eval "db.runCommand({ping:1})"
docker exec mongo2 mongosh --quiet --eval "db.runCommand({ping:1})"
docker exec mongo3 mongosh --quiet --eval "db.runCommand({ping:1})"
```



Capture C2: les trois instances répondent `{ ok: 1 }` à la commande `ping`.

Explication : la commande `ping` ne fait rien d'autre que demander au serveur s'il répond. Un `ok: 1` prouve que chaque instance accepte des connexions et exécute des commandes.

3.3 Configuration en jeu de réplicas

Commandes (dans `docker exec -it mongo1 mongosh`) :

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "mongo1:27017" },
    { _id: 1, host: "mongo2:27017" },
    { _id: 2, host: "mongo3:27017" }
  ]
})
rs.conf()
```

```
test> rs.initiate({ _id: "rs0", members: [ { _id: 0, host: "mongo1:27017" }, { _id: 1, host: "mongo2:27017" }, { _id: 2, host: "mongo3:27017" } ] })
{ ok: 1 }
rs0 [direct: other] test>
```

Capture C3: `rs.initiate` renvoie `{ ok: 1 }` ; le prompt passe à `rs0 [direct: other]` le temps de la première élection.

```
rs0 [direct: other] test> rs.conf()
{
  _id: 'rs0',
  version: 1,
  term: 1,
  members: [
    {
      _id: 0,
      host: 'mongo1:27017',
      arbiterOnly: false,
      buildIndexes: true,
      hidden: false,
      priority: 1,
      tags: {},
      secondaryDelaySecs: Long('0'),
      votes: 1
    },
    {
      _id: 1,
      host: 'mongo2:27017',
      arbiterOnly: false,
      buildIndexes: true,
      hidden: false,
      priority: 1,
      tags: {},
      secondaryDelaySecs: Long('0'),
      votes: 1
    },
    {
      _id: 2,
      host: 'mongo3:27017',
      arbiterOnly: false,
      buildIndexes: true,
      hidden: false,
      priority: 1,
      tags: {},
      secondaryDelaySecs: Long('0'),
      votes: 1
    }
  ],
  protocolVersion: Long('1'),
  writeConcernMajorityJournalDefault: true,
  settings: {
    chainingAllowed: true,
    heartbeatIntervalMillis: 2000,
    heartbeatTimeoutSecs: 10,
    electionTimeoutMillis: 10000,
    catchUpTimeoutMillis: -1,
    catchUpTakeoverDelayMillis: 30000,
    getLastErrorModes: {},
    getLastErrorDefaults: { w: 1, wtimeout: 0 },
    replicaSetId: ObjectId('6acb8625b50e0b62666fc8bd')
  }
}
```

Capture C4: `rs.conf()` : jeu `rs0`, trois membres de priorité 1 avec un vote chacun, `heartbeatIntervalMillis: 2000` et `electionTimeoutMillis: 10000`.

Explication : `rs.initiate` indique à mongo1 la liste des membres et le nom du jeu de réplicas. Les instances s'adressent entre elles par leur nom de conteneur (`mongo1` , `mongo2` , `mongo3`), que Docker résout sur le réseau `mongo-net` . Les membres échangent leur configuration, puis procèdent à une première élection. `rs.conf()` affiche la configuration prise en compte : on y retrouve les trois membres avec un identifiant, une adresse, et une priorité identique (1) qui les rend tous éligibles au rôle de primaire.

3.4 Identification du primaire et des secondaires

Commande :

```
rs.status().members.map(m => ({ nom: m.name, etat: m.stateStr }))
```

```
rs0 [direct: primary] test> rs.status().members.map(m ⇒ ({ nom: m.name, etat: m.stateStr }))
[
  { nom: 'mongo1:27017', etat: 'PRIMARY' },
  { nom: 'mongo2:27017', etat: 'SECONDARY' },
  { nom: 'mongo3:27017', etat: 'SECONDARY' }
]
```

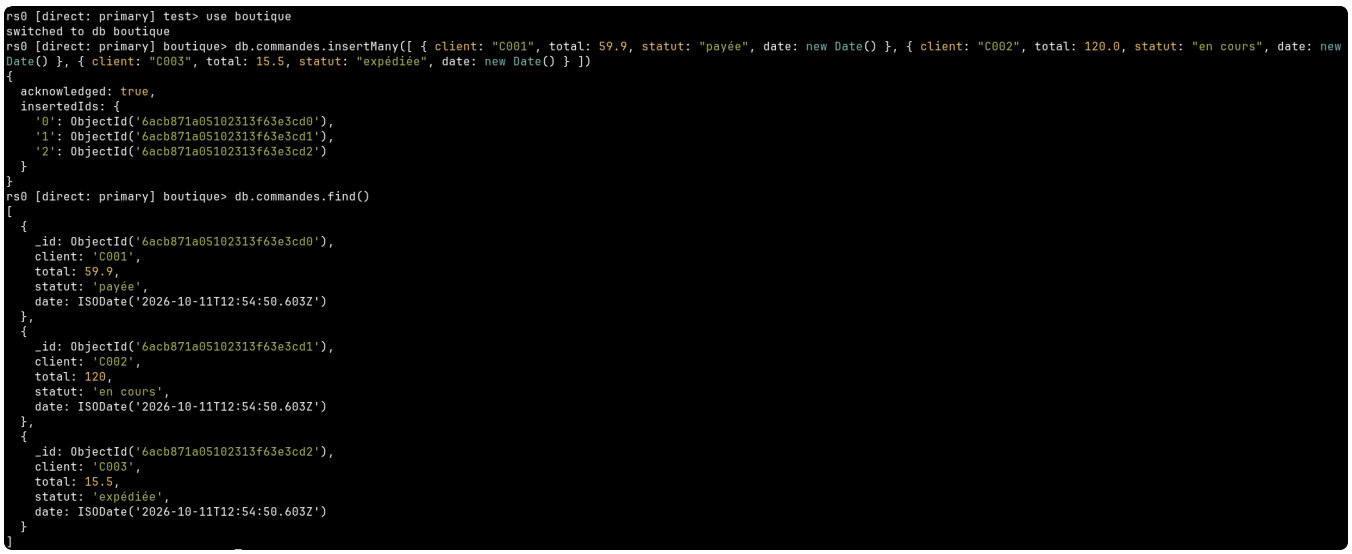
Capture C5: mongo1 est `PRIMARY`, mongo2 et mongo3 sont `SECONDARY`.

Explication : `rs.status()` décrit l'état de chaque membre tel que le voit le jeu de réplicas. Le primaire initial est mongo1 ; mongo2 et mongo3 sont secondaires. C'est le résultat de la première élection, organisée automatiquement après `rs.initiate` : le prompt de `mongosh` est d'ailleurs passé de `rs0 [direct: other]` (capture C3) à `rs0 [direct: primary]` (capture C5).

3.5 Insertion de documents sur le primaire

Commandes :

```
use boutique
db.commandes.insertMany([
  { client: "C001", total: 59.9,  statut: "payée",    date: new Date() },
  { client: "C002", total: 120.0, statut: "en cours", date: new Date() },
  { client: "C003", total: 15.5,  statut: "expédiée", date: new Date() }
])
db.commandes.find()
```



Capture C7: `insertMany` acquitté (`acknowledged: true`, trois identifiants) puis `find()` sur le primaire mongo1.

Explication : nous créons la base `boutique` et la collection `commandes` en y insérant trois documents. L'écriture est faite sur le primaire, seul membre autorisé à en accepter. Le primaire inscrit l'opération dans son journal (l'oplog) ; c'est ce journal que les secondaires vont consulter pour se mettre à jour.

3.6 Vérification de la réplication sur un secondaire

Commandes (dans `docker exec -it mongo2 mongosh`, si mongo2 est secondaire) :

```
use boutique
db.commandes.find()
```

```
rs0 [direct: secondary] test> use boutique
switched to db boutique
rs0 [direct: secondary] boutique> db.commandes.find()
[
  {
    _id: ObjectId('6acb871a05102313f63e3cd0'),
    client: 'C001',
    total: 59.9,
    statut: 'payée',
    date: ISODate('2026-10-11T12:54:50.603Z')
  },
  {
    _id: ObjectId('6acb871a05102313f63e3cd1'),
    client: 'C002',
    total: 120,
    statut: 'en cours',
    date: ISODate('2026-10-11T12:54:50.603Z')
  },
  {
    _id: ObjectId('6acb871a05102313f63e3cd2'),
    client: 'C003',
    total: 15.5,
    statut: 'expédiée',
    date: ISODate('2026-10-11T12:54:50.603Z')
  }
]
rs0 [direct: secondary] boutique>
```

Capture C8: sur un secondaire (prompt `rs0 [direct: secondary]`), les trois commandes sont présentes.

Explication : les documents insérés sur le primaire apparaissent sur le secondaire sans que nous les y ayons écrits : ils ont été copiés par réplication. La réplication est asynchrone, avec un délai généralement très court. Les lectures sur un secondaire sont possibles mais peuvent, en théorie, renvoyer des données légèrement en retard.

3.7 Tentative d'écriture sur un secondaire

Commande (sur le secondaire) :

```
db.commandes.insertOne({ client: "C999", total: 1, statut: "test" })
```

```
rs0 [direct: secondary] boutique> db.commandes.insertOne({ client: "C999", total: 1, statut: "test" })
MongoServerError[NotWritablePrimary]: not primary
rs0 [direct: secondary] boutique>
```

Capture C9: l'insertion directe sur le secondaire est refusée : `NotWritablePrimary`, « not primary ».

Message relevé : `MongoServerError[NotWritablePrimary]: not primary`.

Explication : le secondaire refuse l'écriture et indique qu'il n'est pas le primaire. Cette règle est voulue (voir question 2, section 6) : une seule instance reçoit les écritures, afin que toutes les copies restent identiques.

3.8 Arrêt du primaire

Pour observer précisément la bascule, nous avons utilisé un script qui interroge mongo1, mongo2 et mongo3 et affiche le rôle que chaque instance déclare (`watch.sh`). Chaque passage dure en réalité 1,5 à 2 secondes, le temps de lancer `mongosh` dans chaque conteneur. Une seule ligne de commande a servi à noter l'heure, supprimer brutalement le conteneur primaire pour simuler une panne, puis lancer le script :

```
date +%T ; docker kill mongo1; ./watch.sh
```

(Remplacer `mongo1` par le primaire réel.) Nous avons choisi `docker kill` plutôt que `docker stop` : `docker stop` arrête MongoDB proprement, qui cède alors son rôle immédiatement et rend l'élection quasi instantanée, alors que `docker kill` reproduit une vraie panne où les secondaires doivent la détecter eux-mêmes.

Nous avons réalisé la manipulation deux fois (essai n°1 à 13:03, essai n°2 à 13:17). Entre les deux, `watch.sh` a été modifié pour afficher aussi mongo1, qui devient `injoignable` après le `kill`.

Essai n°2 (13:17)

```
./watch.sh
13:16:28  mongo1=PRIMARY   mongo2=SECONDARY   mongo3=SECONDARY
13:16:30  mongo1=PRIMARY   mongo2=SECONDARY   mongo3=SECONDARY
13:16:32  mongo1=PRIMARY   mongo2=SECONDARY   mongo3=SECONDARY
13:16:34  mongo1=PRIMARY   mongo2=SECONDARY   ^C⏎
```

Capture C10: état stable avant la panne : mongo1 `PRIMARY`, mongo2 et mongo3 `SECONDARY`.

```
› MongoDB_Repl&Sharding
date +%T ; docker kill mongo1; ./watch.sh
13:17:07
mongo1
13:17:07  mongo1=injoignable   mongo2=SECONDARY   mongo3=SECONDARY
13:17:09  mongo1=injoignable   mongo2=SECONDARY   mongo3=SECONDARY
13:17:10  mongo1=injoignable   mongo2=SECONDARY   mongo3=SECONDARY
13:17:12  mongo1=injoignable   mongo2=SECONDARY   mongo3=SECONDARY
13:17:13  mongo1=injoignable   mongo2=SECONDARY   mongo3=SECONDARY
13:17:15  mongo1=injoignable   mongo2=SECONDARY   mongo3=SECONDARY
13:17:17  mongo1=injoignable   mongo2=SECONDARY   mongo3=PRIMARY
13:17:18  mongo1=injoignable   mongo2=SECONDARY   mongo3=PRIMARY
13:17:20  mongo1=injoignable   mongo2=SECONDARY   mongo3=PRIMARY
13:17:21  mongo1=injoignable   mongo2=SECONDARY   mongo3=PRIMARY
^C⏎
```

Capture C11: bascule : `docker kill mongo1` à 13:17:07, aucun primaire jusqu'à 13:17:15, mongo3 `PRIMARY` à 13:17:17.

Essai n°1 (13:03)

```
› MongoDB_Repl&Sharding
date +%T ; docker kill mongo1; ./watch.sh
13:03:23
mongo1
13:03:23  mongo2=SECONDARY   mongo3=SECONDARY
13:03:25  mongo2=SECONDARY   mongo3=SECONDARY
13:03:27  mongo2=SECONDARY   mongo3=SECONDARY
13:03:28  mongo2=SECONDARY   mongo3=SECONDARY
13:03:30  mongo2=SECONDARY   mongo3=SECONDARY
13:03:31  mongo2=SECONDARY   mongo3=SECONDARY
13:03:33  mongo2=SECONDARY   mongo3=SECONDARY
13:03:35  mongo2=PRIMARY   mongo3=SECONDARY
13:03:36  mongo2=PRIMARY   mongo3=SECONDARY
13:03:38  mongo2=PRIMARY   mongo3=SECONDARY
13:03:40  mongo2=PRIMARY   mongo3=SECONDARY
```

Capture C11 bis: essai n°1 : `docker kill mongo1` à 13:03:23, mongo2 `PRIMARY` à 13:03:35 (le script n'affichait pas encore mongo1).

Explication : juste après l'arrêt, le primaire n'existe plus. Dans l'essai n°2, `docker kill` est lancé à 13:17:07 : mongo1 est aussitôt `injoignable`, et mongo2 comme mongo3 restent `SECONDARY` jusqu'à 13:17:15. Pendant ce temps aucun membre ne peut accepter d'écriture,

car les secondaires n'ont pas encore constaté la disparition du primaire. À 13:17:17, mongo3 apparaît `PRIMARY` : il a été élu. Dans l'essai n°1 (arrêt à 13:03:23), c'est mongo2 qui devient primaire à 13:03:35. L'analyse détaillée est en section 4.

3.9 Nouveau primaire et accès aux données

Commandes (sur le nouveau primaire) :

```
rs.status().members.map(m => ({ nom: m.name, etat: m.stateStr }))
use boutique
db.commandes.find()
db.commandes.insertOne({ client: "C004", total: 42, statut: "payée" })
```

```
rs0 [direct: primary] test> rs.status().members.map(m ⇒ ({ nom: m.name, etat: m.stateStr }))
[
  { nom: 'mongo1:27017', etat: '(not reachable/healthy)' },
  { nom: 'mongo2:27017', etat: 'PRIMARY' },
  { nom: 'mongo3:27017', etat: 'SECONDARY' }
]
```

Capture C12: `rs.status()` après l'essai n°1 : mongo1 `(not reachable/healthy)`, mongo2 `PRIMARY`, mongo3 `SECONDARY`.

```
rs0 [direct: primary] test> use boutique
switched to db boutique
rs0 [direct: primary] boutique> db.commandes.find()
[
  {
    _id: ObjectId('6acb871a05102313f63e3cd0'),
    client: 'C001',
    total: 59.9,
    statut: 'payée',
    date: ISODate('2026-10-11T12:54:50.603Z')
  },
  {
    _id: ObjectId('6acb871a05102313f63e3cd1'),
    client: 'C002',
    total: 120,
    statut: 'en cours',
    date: ISODate('2026-10-11T12:54:50.603Z')
  },
  {
    _id: ObjectId('6acb871a05102313f63e3cd2'),
    client: 'C003',
    total: 15.5,
    statut: 'expédiée',
    date: ISODate('2026-10-11T12:54:50.603Z')
  }
]
rs0 [direct: primary] boutique> db.commandes.insertOne({ client: "C004", total: 42, statut: "payée" })
{
  acknowledged: true,
  insertedId: ObjectId('6acb89d62da8226d88250495')
}
```

Capture C13: sur le nouveau primaire, les trois commandes sont intactes et l'insertion de C004 est acceptée.

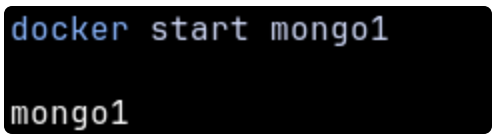
Explication : à l'issue de l'essai n°1, le nouveau primaire est mongo2 : `rs.status()` affiche mongo1 comme `(not reachable/healthy)`, mongo2 `PRIMARY` et mongo3 `SECONDARY` (capture C12). Les trois commandes initiales sont toujours là, car mongo2 en détenait déjà une copie, et l'insertion de la commande C004 est acceptée (capture C13) : le service a repris sans intervention humaine. C'est la démonstration de la haute disponibilité.

3.10 Redémarrage de l'ancienne instance primaire

Commandes :

```
docker start mongo1
docker exec -it mongo1 mongosh
```

```
rs.status().members.map(m => ({ nom: m.name, etat: m.stateStr }))
use boutique
db.commandes.find()
```



Capture C15 (a): `docker start mongo1`.

```
rs0 [direct: secondary] test> rs.status().members.map(m => ({ nom: m.name, etat: m.stateStr }))
[
  { nom: 'mongo1:27017', etat: 'SECONDARY' },
  { nom: 'mongo2:27017', etat: 'PRIMARY' },
  { nom: 'mongo3:27017', etat: 'SECONDARY' }
]
rs0 [direct: secondary] test> use boutique
switched to db boutique
rs0 [direct: secondary] boutique> db.commandes.find()
[
  {
    _id: ObjectId('6acb871a05102313f63e3cd0'),
    client: 'C001',
    total: 59.9,
    statut: 'payée',
    date: ISODate('2026-10-11T12:54:50.603Z')
  },
  {
    _id: ObjectId('6acb871a05102313f63e3cd1'),
    client: 'C002',
    total: 120,
    statut: 'en cours',
    date: ISODate('2026-10-11T12:54:50.603Z')
  },
  {
    _id: ObjectId('6acb871a05102313f63e3cd2'),
    client: 'C003',
    total: 15.5,
    statut: 'expédiée',
    date: ISODate('2026-10-11T12:54:50.603Z')
  },
  {
    _id: ObjectId('6acb89d62da8226d88250495'),
    client: 'C004',
    total: 42,
    statut: 'payée'
  }
]
```

Capture C15 (b): mongo1 est revenu en `SECONDARY` (mongo2 reste `PRIMARY`) et la commande C004, insérée pendant son absence, est présente.

Explication : l'instance redémarrée ne reprend pas son ancien rôle : un nouveau primaire existe déjà, elle le détecte et se rattache à lui en tant que secondaire. Elle rattrape les écritures qu'elle a manquées pendant son absence (ici la commande C004) en lisant l'oplog du nouveau primaire.

4. Analyse de l'élection

4.1 Mesures

Mesure	Essai n°1	Essai n°2
Primaire avant la panne	mongo1	mongo1
Heure de l'arrêt (`docker kill mongo1`)	13:03:23	13:17:07
Premier PRIMARY observé par `watch.sh`	mongo2 à 13:03:35	mongo3 à 13:17:17
Durée sans primaire (mesurée, à ± 2 s)	≈ 12 s	≈ 10 s
Début de l'élection d'après les journaux	13:03:33.829 (mongo2, mandat 6)	non capturé
Instance élue	mongo2	mongo3

4.2 Comment nous l'avons constaté

Nous avons utilisé deux sources. D'abord, le script `watch.sh` (capture C11), qui affiche toutes les 1,5 à 2 secondes le rôle déclaré par chaque instance : on y lit directement la période sans primaire puis l'apparition du nouveau. Ensuite, les journaux de l'instance élue, qui contiennent les événements de l'élection avec leur horodatage :

```
docker logs mongo2 2>&1 | grep -E "Starting an election|Election succeeded"
```

Image — Capture C14

Capture C14: journaux de mongo2 et de mongo3 filtrés sur les messages d'élection.

Dans le journal de mongo2, le message « Starting an election, since we've seen no PRIMARY in election timeout period » est horodaté 13:03:33.829 (UTC ; ces heures coïncident avec celles relevées dans le terminal), soit environ 10 à 11 secondes après l'arrêt de mongo1 à 13:03:23. Le message « Election succeeded, assuming primary role »

suit 2 millisecondes plus tard, au mandat 6 : mongo2 est donc l'instance élue lors de l'essai n°1. Les lignes plus anciennes (mongo3 à 13:00:59, mongo2 à 13:01:38 « due to step up request ») correspondent à des essais préalables et ne concernent pas la panne analysée.

4.3 Explication du mécanisme

Les membres d'un jeu de réplicas s'envoient régulièrement des messages de contrôle (battements de cœur, ou heartbeats), par défaut toutes les 2 secondes. Si un secondaire ne reçoit plus de réponse du primaire pendant le délai d'élection (`electionTimeoutMillis`, 10 secondes par défaut), il considère que le primaire est tombé et se porte candidat : il augmente le numéro de mandat (term), vote pour lui-même et demande leur vote aux autres membres. Un membre vote pour un candidat dont les données sont au moins aussi à jour que les siennes. Le candidat qui recueille les voix de la majorité des membres devient primaire et commence à accepter les écritures. Les autres membres le suivent comme secondaires.

Nos mesures (≈ 12 s pour l'essai n°1, ≈ 10 s pour l'essai n°2) sont cohérentes avec le délai théorique de 10 secondes (`electionTimeoutMillis: 10000` et `heartbeatIntervalMillis: 2000`, visibles dans `rs.conf()`, capture C4). Ce délai court à partir du dernier battement de cœur reçu du primaire ; s'y ajoutent un court délai aléatoire propre à chaque membre et le vote lui-même, quasi instantané (2 ms dans le journal de l'essai n°1). L'écart d'environ 2 secondes entre les deux essais s'explique en partie par la précision de `watch.sh`, qui ne sonde les membres que toutes les 1,5 à 2 secondes. Ce n'est pas la même instance qui a gagné dans les deux essais (mongo2, puis mongo3) : les deux secondaires étant à jour et de priorité égale, c'est en pratique le premier à atteindre la fin de son délai qui remporte l'élection.

Si nous avions utilisé `docker stop`, MongoDB aurait reçu un signal d'arrêt propre et aurait transmis volontairement son rôle à un secondaire, ce qui réduit fortement le délai. Ce cas n'a pas été mesuré ici.

4.4 Pourquoi un nombre impair de membres

Pour être élu, un candidat doit obtenir la majorité stricte des voix de l'ensemble du jeu. Avec 3 membres, il faut 2 voix : une panne est tolérée. Avec 4 membres, il en faut 3 : une seule panne est toujours tolérée, donc le quatrième serveur n'apporte aucune résistance supplémentaire. Surtout, avec un nombre pair, une coupure réseau peut séparer le jeu en deux moitiés égales, dont aucune n'a la majorité : plus aucun primaire ne peut être élu. Avec 5 membres, il faut 3 voix : deux pannes sont tolérées.

Membres	Majorité requise	Pannes tolérées
3	2	1
4	3	1
5	3	2

5. Le partitionnement

Cette partie n'a pas été mise en œuvre : elle est traitée à l'écrit, à partir de la documentation officielle de MongoDB.

5.1 Principe

Le partitionnement (sharding) répartit les documents d'une même collection sur plusieurs serveurs appelés fragments (shards). Chaque fragment ne contient qu'une partie des données. La capacité de stockage et le débit d'écriture augmentent en ajoutant des fragments : c'est un passage à l'échelle horizontal, par opposition à l'achat d'une machine toujours plus puissante (passage à l'échelle vertical), qui finit par atteindre une limite physique et financière.

5.2 Architecture d'un cluster partitionné

Un cluster partitionné comprend trois types de composants :

- **Les fragments (shards)** stockent les données. En production, chaque fragment est lui-même un jeu de réplicas, ce qui combine partitionnement et haute disponibilité.
- **Les serveurs de configuration** conservent les métadonnées du cluster : quelles plages de valeurs de la clé de partitionnement se trouvent sur quel fragment. Ils forment eux-mêmes un jeu de réplicas.
- **Le routeur `mongos`** est le point d'entrée de l'application. Il consulte les métadonnées et dirige chaque requête vers le ou les fragments concernés, puis rassemble les résultats. L'application se connecte à `mongos` comme à un serveur MongoDB ordinaire.

Les données sont découpées en morceaux (chunks), des plages contiguës de la clé de partitionnement. Un processus d'équilibrage (balancer) déplace des morceaux d'un fragment à l'autre pour que la répartition reste homogène.

> **[SCHÉMA — à insérer ici : application, mongos, serveurs de configuration (jeu de réplicas), trois fragments (chacun un jeu de réplicas)]** Il est possible de reprendre la slide d'architecture de la présentation.

5.3 La clé de partitionnement

La clé de partitionnement est le ou les champs d'un document qui déterminent le fragment où il est stocké. Elle est choisie lors de la mise en place du partitionnement d'une collection et conditionne durablement les performances. MongoDB propose deux grandes stratégies :

- **Partitionnement par plages** : les documents sont regroupés selon des plages de valeurs de la clé. Les requêtes par intervalle sont efficaces, mais une clé dont les valeurs croissent régulièrement concentre toutes les nouvelles insertions sur un seul fragment.
- **Partitionnement par hachage** : MongoDB applique une fonction de hachage à la valeur de la clé, ce qui répartit les documents de manière uniforme. Les requêtes d'égalité sur la clé sont efficaces, mais les requêtes par intervalle doivent interroger tous les fragments.

Une bonne clé a une forte cardinalité (beaucoup de valeurs différentes), répartit les écritures de façon homogène et apparaît dans les requêtes les plus fréquentes, afin que `mongos` puisse cibler un seul fragment. Un mauvais choix a trois conséquences possibles : un fragment surchargé pendant que les autres restent presque inactifs (hotspot), des morceaux qui ne peuvent plus être divisés parce qu'ils partagent tous la même valeur, et des requêtes qui doivent interroger tous les fragments.

5.4 Choix pour la plateforme de commerce

Pour la collection `commandes`, nous recommandons la clé `{ customerId: "hashed" }`. Le nombre de clients est élevé, ce qui donne une forte cardinalité ; le hachage répartit les commandes uniformément entre les fragments ; et la requête la plus courante d'une boutique (« les commandes de ce client ») est une requête d'égalité sur `customerId`, ciblée sur un seul fragment. Les requêtes par période (par exemple, les commandes du jour pour la comptabilité) interrogeront tous les fragments, ce que nous jugeons acceptable pour un usage moins fréquent.

Une alternative serait la clé composée `{ customerId: 1, orderDate: 1 }`, qui garde les commandes d'un client regroupées et triées par date.

```
sh.enableSharding("boutique")
sh.shardCollection("boutique.commandes", { customerId: "hashed" })
```

Ces commandes sont données à titre d'illustration : elles n'ont pas été exécutées dans cette simulation.

Clés à éviter :

- `orderDate` ou `_id` (un ObjectId contient un horodatage) : leurs valeurs croissent avec le temps, donc toutes les nouvelles commandes arrivent sur le fragment qui détient la plage la plus récente. Ce fragment sature pendant que les autres sont sous-utilisés, ce qui annule le bénéfice du partitionnement.
- `statut` : très peu de valeurs différentes (payée, en cours, expédiée…). Le nombre de fragments utiles est plafonné par le nombre de valeurs, et tous les documents ayant le même statut forment des morceaux indivisibles.

6. Réponses aux questions

Question 1: Que se passe-t-il si deux membres sur trois deviennent indisponibles ?

Le membre restant ne peut plus obtenir la majorité : pour être primaire, il faudrait 2 voix sur 3, et il n'en a qu'une. Si c'était le primaire, il se rétrograde de lui-même en secondaire, et aucun nouveau primaire ne peut être élu. Le jeu de réplicas n'accepte donc plus aucune écriture. Les lectures restent possibles sur le membre restant si l'application accepte de lire sur un secondaire, avec le risque de données légèrement en retard. Dès qu'un second membre revient, la majorité est rétablie, une élection a lieu et les écritures reprennent.

[À COMPLÉTER si le test bonus a été réalisé : insérer ici la capture C16 (état du membre restant en `SECONDARY`) et le message d'erreur reçu lors d'une tentative d'écriture.]

Question 2: Pourquoi une écriture sur un secondaire est-elle refusée, alors que la lecture y est possible ?

MongoDB utilise un modèle à écrivain unique : le primaire ordonne toutes les écritures dans son journal, et les secondaires ne font que rejouer ce journal dans le même ordre. Si un secondaire acceptait une écriture directe, elle n'apparaîtrait pas dans le journal du primaire : les copies divergeraient et il faudrait ensuite résoudre des conflits entre deux versions d'un même document. Refuser l'écriture garantit que toutes les copies convergent vers le même état.

La lecture, elle, ne modifie rien : elle peut être servie sans risque par n'importe quelle copie. Elle permet de répartir la charge de lecture, au prix d'un éventuel retard de réplication : un secondaire peut renvoyer une donnée qui n'est pas encore la plus récente.

Question 3: Quelle clé de partitionnement pour la collection des commandes, et laquelle éviter ?

Nous choisirions `{ customerId: "hashed" }` (ou la clé composée `{ customerId: 1, orderDate: 1 }`), pour les raisons exposées en section 5.4. Il faut absolument éviter une clé monotone comme `orderDate` ou `_id` : tous les nouveaux documents iraient sur le même fragment, qui deviendrait un goulot d'étranglement. Il faut aussi éviter une clé à faible cardinalité comme `statut`.

Question 4: Le partitionnement et la réplication répondent-ils au même besoin ? Peut-on employer les deux ensemble ?

Non. La réplication répond à un besoin de disponibilité et de durabilité : chaque donnée existe en plusieurs exemplaires, donc la panne d'un serveur n'arrête pas le service. Elle n'augmente pas la capacité d'écriture, puisque tous les membres contiennent toutes les données et que seul le primaire écrit. Le partitionnement répond à un besoin de volume et de débit : chaque serveur ne gère qu'une partie des données. Il ne protège pas, à lui seul, d'une panne : si un fragment unique tombe, la partie des données qu'il détient devient inaccessible.

Les deux se combinent, et c'est l'architecture standard en production : chaque fragment est un jeu de réplicas, tout comme les serveurs de configuration. Le partitionnement fournit la capacité, la réplication fournit la résistance aux pannes.

7. Conclusion et recommandation

La simulation a montré qu'un jeu de réplicas de trois instances supprime le point de défaillance unique : après l'arrêt brutal du primaire, un secondaire a été élu en environ 10 à 12 secondes (sur deux essais), sans intervention humaine, et les données sont restées accessibles. Nous avons aussi vérifié que les secondaires refusent les écritures et que l'ancienne instance primaire revient comme secondaire.

Pour la plateforme de commerce, nous recommandons :

1. **Immédiatement** : migrer vers un jeu de réplicas de trois membres, placés sur des machines (ou des zones) distinctes. C'est la mesure la moins coûteuse, et elle règle le problème de la panne et de la perte de commandes. Les applications doivent utiliser une chaîne de connexion qui liste le jeu de réplicas pour suivre automatiquement le changement de primaire.
2. **Lorsque le volume de données ou le débit d'écriture dépasse la capacité d'un seul serveur** : partitionner la collection des commandes avec la clé `{ customerId: "hashed" }`, en conservant des jeux de réplicas pour chaque fragment.

On évite ainsi de payer trop tôt la complexité du partitionnement, tout en préparant la croissance. En termes de théorème CAP, MongoDB retient la cohérence et la tolérance aux partitions réseau : en cas de coupure, la partie du cluster qui n'a pas la majorité refuse les écritures plutôt que de laisser les données diverger. Pour un site de commerce, où une commande ne doit ni être perdue ni dupliquée, c'est le bon compromis.

8. Sources

- MongoDB Manual — Replication : https://www.mongodb.com/docs/manual/replication/
- MongoDB Manual — Replica Set Elections : https://www.mongodb.com/docs/manual/core/replica-set-elections/
- MongoDB Manual — Replica Set Members : https://www.mongodb.com/docs/manual/core/replica-set-members/
- MongoDB Manual — Replica Set Oplog : https://www.mongodb.com/docs/manual/core/replica-set-oplog/
- MongoDB Manual — Sharding : https://www.mongodb.com/docs/manual/sharding/
- MongoDB Manual — Shard Keys : https://www.mongodb.com/docs/manual/core/sharding-shard-key/
- MongoDB Manual — Sharded Cluster Components : https://www.mongodb.com/docs/manual/core/sharded-cluster-components/
- Image Docker officielle `mongo` : https://hub.docker.com/_/mongo
- Gilbert S. et Lynch N. (2002), Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services, ACM SIGACT News.
- Support de cours M201 — Comprendre les bases de données NoSQL (S. Abderrahman).